NODE.JS ECOSYSTEM & REAL-TIME CHAT APPLICATION

Assignment Report

Node.js • Express.js • MongoDB • Socket.io • Mocha • Chai

GitHub Repository

https://github.com/khushipg04-tech/version-control-testing-project

This report documents the concepts, setup process, application structure, database integration, real-time communication, testing workflow, and GitHub submission process.

1. Introduction

Node.js is a JavaScript runtime used to build server-side applications. Its event-driven, non-blocking I/O model is useful for web servers, APIs and real-time applications. This assignment explores the Node.js ecosystem and applies it to a web application and a real-time chat application.

The project uses Express.js for HTTP routing, MongoDB for data persistence, Socket.io for real-time communication, and Mocha/Chai for testing.

2. Objectives

• Understand Node.js architecture and npm.

• Set up a Node.js development environment.

• Create a basic Express.js web application.

• Understand MongoDB integration.

• Build real-time communication with Socket.io.

• Implement automated testing with Mocha and Chai.

• Manage and submit the project through Git and GitHub.

3. Node.js Architecture

Node.js executes JavaScript outside the browser using the V8 engine. Its event loop coordinates asynchronous work, allowing I/O operations such as network and database access without blocking the main JavaScript execution flow.

Important concepts include the V8 engine, event loop, callbacks, Promises, async/await, npm packages and non-blocking I/O.

```
const fs = require('fs');

fs.readFile('data.txt', 'utf8', (err, data) => {
    if (err) return console.error(err);
    console.log(data);
});
console.log('Execution continues while the file is read.');
```

4. Environment Setup

Install Node.js, then verify Node.js and npm from PowerShell or Command Prompt.

```
node -v
npm -v
```

Initialize a project and install the required packages:

```
npm init -y
npm install express mongoose socket.io
npm install --save-dev mocha chai
```

If PowerShell reports that npm.ps1 cannot be loaded because script execution is disabled, npm.cmd can be used, for example: npm.cmd install.

5. Express.js Web Application

Express.js is a lightweight Node.js framework for web applications and APIs. It provides routing and middleware support.

```
const express = require('express');

const app = express();
```

```
app.use(express.json());

app.get('/', (req, res) => {
    res.send('Node.js Express application is running');
});

app.listen(3000, () => {
    console.log('Server running on port 3000');
});
```

6. MongoDB Integration

MongoDB is a NoSQL document database with a flexible schema. Mongoose is commonly used with Node.js to define schemas and interact with MongoDB.

```
const mongoose = require('mongoose');

mongoose.connect(process.env.MONGODB_URI)
  .then(() => console.log('MongoDB connected'))
  .catch(err => console.error(err));

const messageSchema = new mongoose.Schema({
    username: String,
    text: String,
    createdAt: { type: Date, default: Date.now }
});

const Message = mongoose.model('Message', messageSchema);
```

The database connection string should be stored in an environment variable and not committed to GitHub.

7. Real-Time Chat with Socket.io

Socket.io enables real-time, bidirectional communication between clients and a server. It is useful for chat applications, notifications and live dashboards.

```
const http = require('http');
const express = require('express');
const { Server } = require('socket.io');

const app = express();
const server = http.createServer(app);
const io = new Server(server);

io.on('connection', (socket) => {
    console.log('User connected');

    socket.on('chat message', (message) => {
        io.emit('chat message', message);
    });

    socket.on('disconnect', () => {
        console.log('User disconnected');
    });
});

server.listen(3000);
```

The io.emit() method broadcasts an event to all connected clients, allowing messages to appear without refreshing the page.

8. Chat Application Flow

Step	Process
1	User opens the chat application.
2	Client establishes a Socket.io connection.
3	User enters a message.

4	Client emits the message to the server.
5	Server receives the event.
6	Server broadcasts the message with io.emit().
7	Connected clients display the message.
8	Message may be stored in MongoDB for chat history.

9. Testing with Mocha and Chai

Mocha is a JavaScript testing framework commonly used with Node.js. Chai provides assertion styles for checking expected results.

```
const { expect } = require('chai');

describe('Basic application tests', () => {
    it('adds two numbers correctly', () => {
        expect(2 + 2).to.equal(4);
    });
});
```

After configuring the test script in package.json, tests can be run with:

```
npm test
```

Testing can cover routes, validation, database operations, message handling and error conditions.

10. Debugging

Node.js applications can be debugged using Visual Studio Code or Chrome DevTools. The Node.js inspector can be enabled with:

```
node --inspect app.js
```

A debugging session allows inspection of the call stack and variables. The debugger statement can pause execution at a chosen point.

11. Git and GitHub

Git tracks source-code changes and GitHub hosts the repository. The project repository is:

https://github.com/khushipg04-tech/version-control-testing-project

A typical workflow is:

```
git status
git add .
git commit -m "Complete Node.js assignment"
git push -u origin feature-testing
```

The repository should contain the project source code and configuration files. Database passwords, tokens and other secrets should not be committed.

12. Learning Outcomes

• Understood Node.js server-side architecture.

• Learned Express.js routes and middleware.

• Understood MongoDB's flexible document model.

• Learned Socket.io real-time communication.

• Learned automated testing with Mocha and Chai.

• Practiced Git/GitHub version control.

13. Conclusion

This assignment provided practical exposure to the Node.js ecosystem. Express.js simplifies web-server development, MongoDB provides flexible data storage, Socket.io enables real-time communication, and Mocha with Chai supports automated testing. Together, these technologies provide a foundation for applications such as real-time chat systems.

The project code is maintained in the GitHub repository linked in this report and the PDF can be submitted through the assignment form.

14. Submission Details

Title	Node.js, Express.js, MongoDB & Socket.io Real-Time Chat Application
Link	https://github.com/khushipg04-tech/version-control-testing-project
Upload	NodeJS_Express_MongoDB_SocketIO_Assignment.pdf